

Weekly Report

General Information

Group ID: 03

Project Name: Yum Recipe

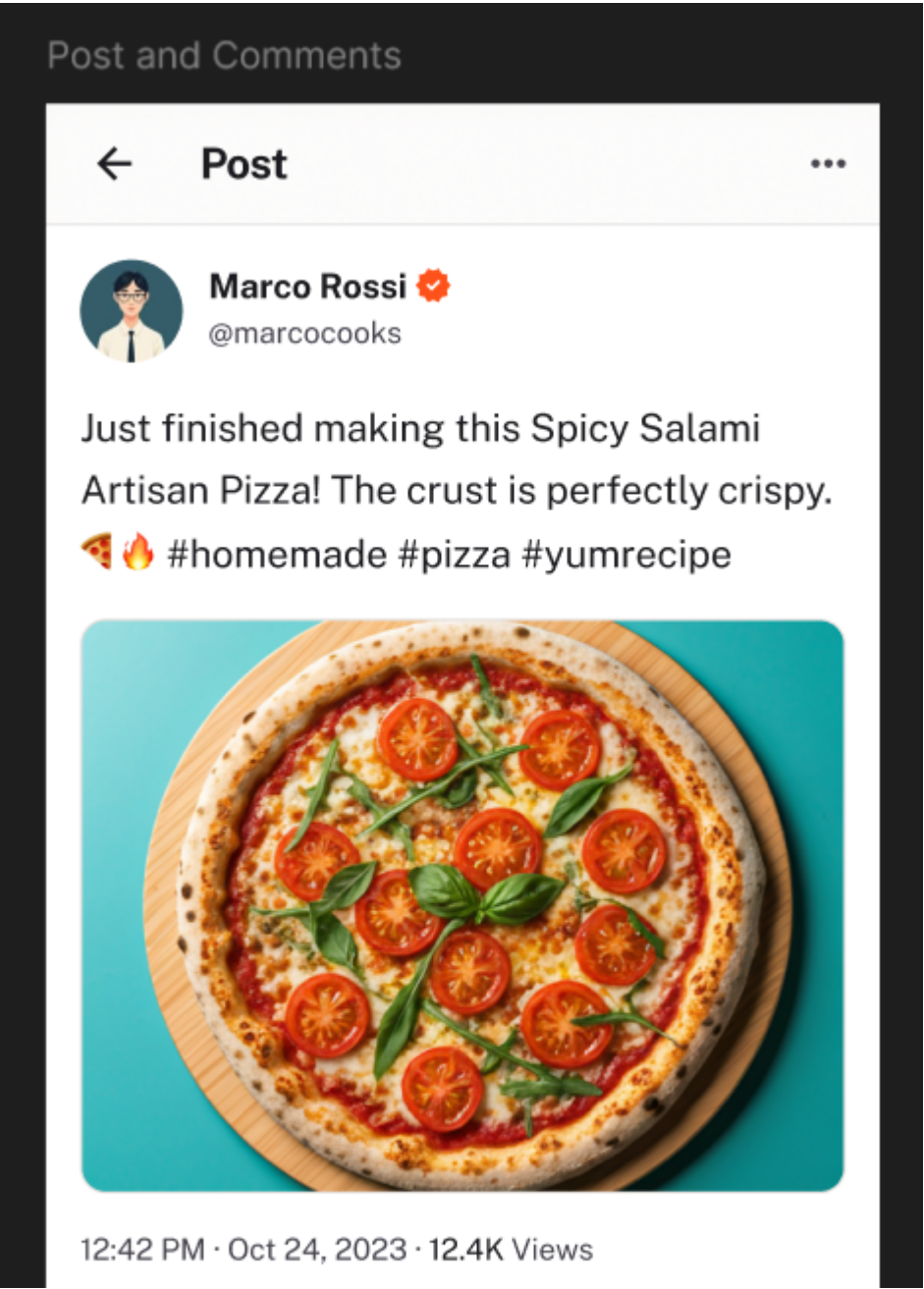
Date Range: 2026-03-08 – 2026-03-14

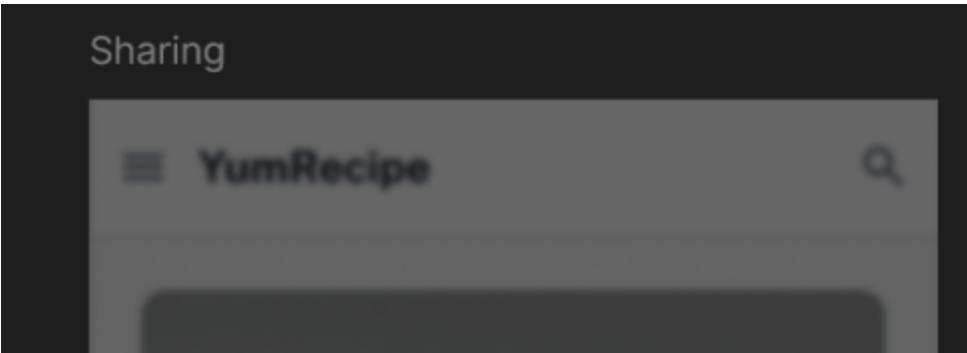
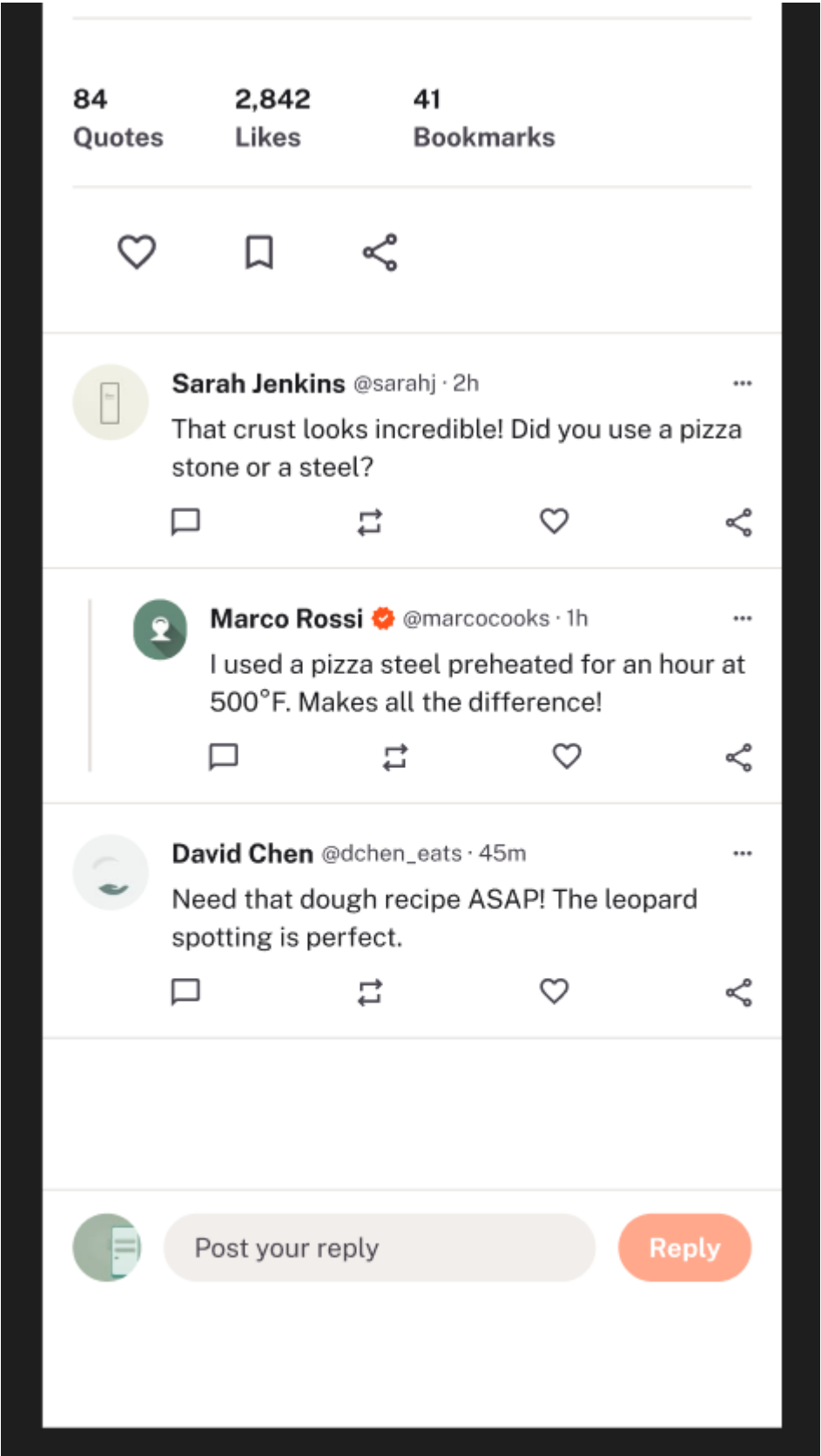
Tasks Completed This Week

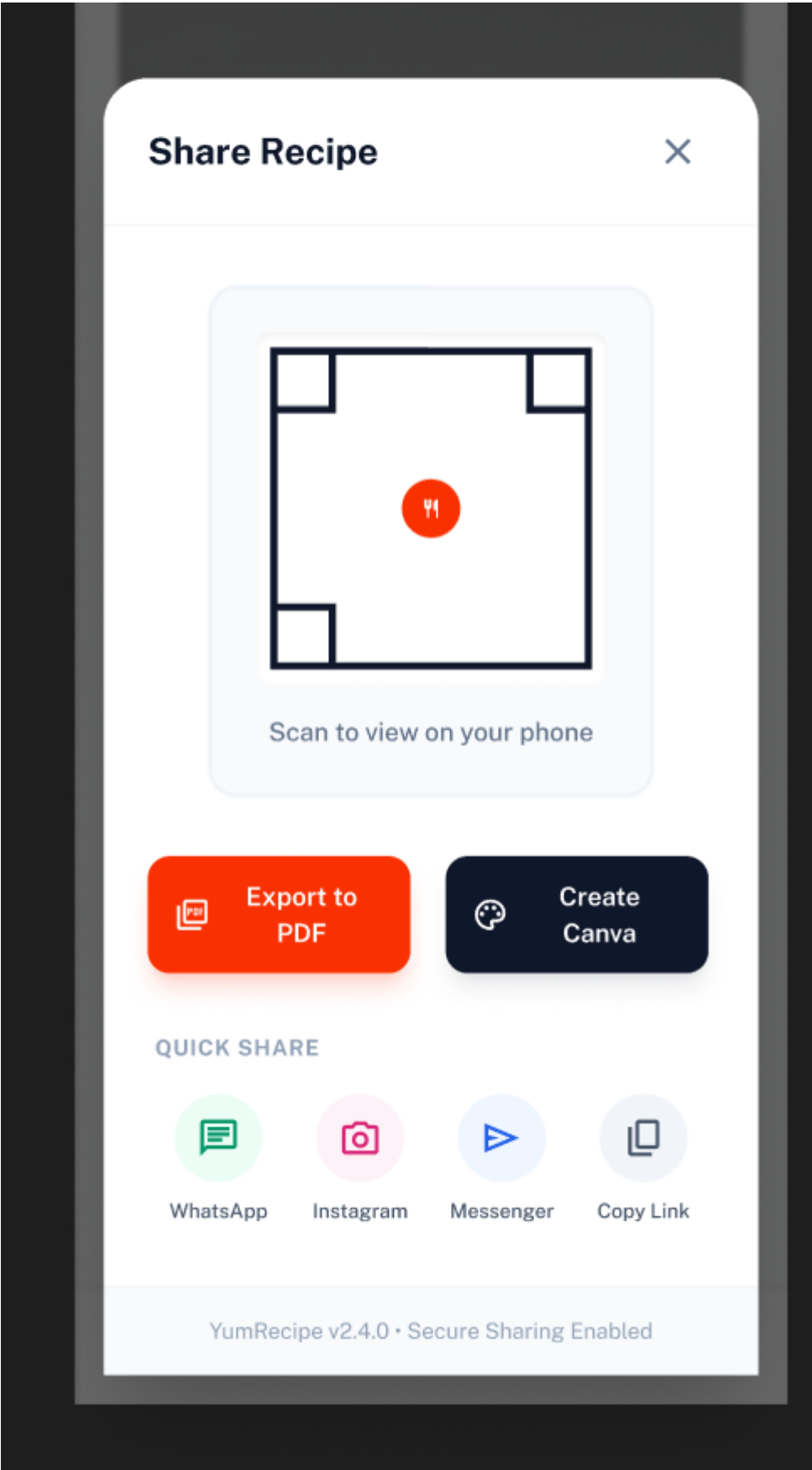
23127122 - Nguyễn Duy Thịnh

- Design the screen for the post view and sharing view.
- Implement the basic home feed screen with dummy data.

► Evidence







- Home feed screen with dummy data:



Welcome back



Le Bonbon

6 hr. ago



Classic Spaghetti Carbonara

A creamy Italian pasta made with eggs, Pecorino Romano, guanciale, and black pepper. No cream needed — the secret is all in the technique!

 142

 28

 Share

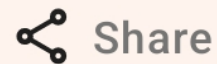


Holo the Wise

Yesterday

Tip: How to keep herbs fresh for 2 weeks

Wrap herbs loosely in a slightly damp paper towel, place them in a zip-lock bag, and store in the refrigerator. Works great for cilantro, parsle...

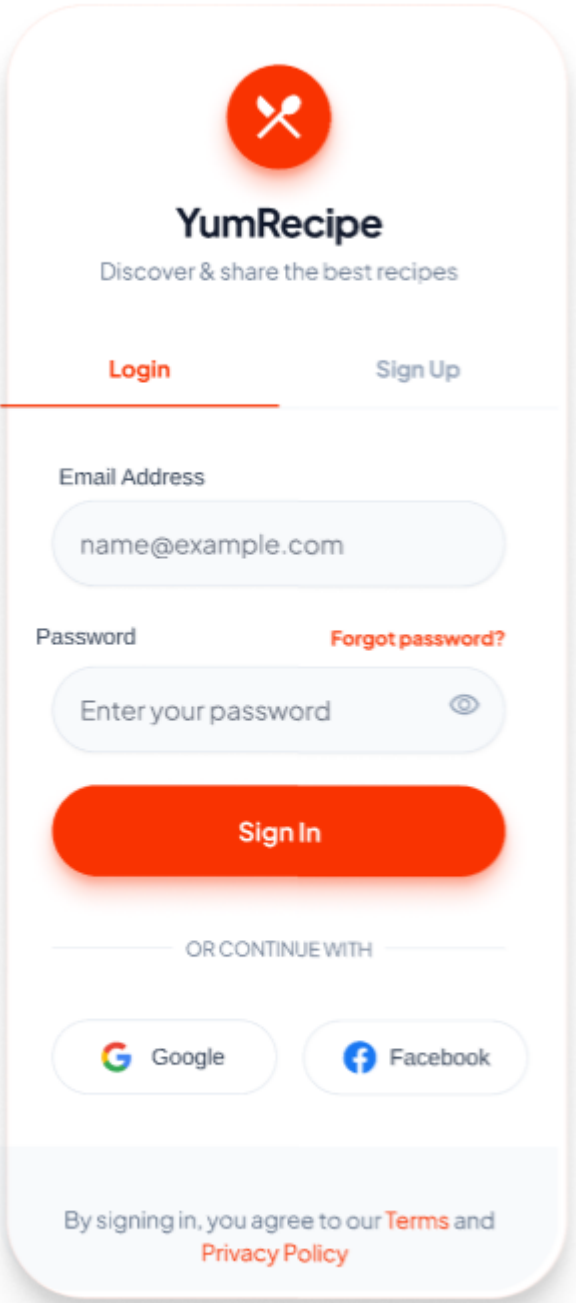


23127205 - Lâm Hữu Khánh

- Design the login, signup, and forgot password screens.
- Implement the APIs for User Registration and Secure Login using JWT (JSON Web Tokens).
- Implement the API and Email Service to handle the Forgot Password and Password Reset requests.
- Implement Unit Tests for authentication controllers and services to ensure system reliability.

► Evidence

- Login screen design:



The image shows a mobile app interface for 'YumRecipe'. At the top is a red circular logo with a white crossed-spoon icon. Below it is the app name 'YumRecipe' in bold, followed by the tagline 'Discover & share the best recipes'. There are two tabs: 'Login' (active, highlighted with a red underline) and 'Sign Up'. The 'Login' section contains an 'Email Address' field with the placeholder 'name@example.com', a 'Password' field with the placeholder 'Enter your password' and a toggle icon, and a 'Forgot password?' link. A large red 'Sign In' button is below the password field. Below this is a section 'OR CONTINUE WITH' with two buttons: 'Google' and 'Facebook'. At the bottom of the main content area is a disclaimer: 'By signing in, you agree to our Terms and Privacy Policy'. The bottom navigation bar has three items: '10K+ RECIPES' with a book icon, 'CHEF NETWORK' with a person icon, and 'TOP RATED' with a star icon.

- Signup screen design:



YumRecipe

Discover & share the best recipes

Login

Sign Up

Full Name

Enter your full name

Email Address

name@example.com

Password

Create a password

Confirm Password

Re-enter your password

Create Account

OR CONTINUE WITH

 Google

 Facebook

By signing up, you agree to our [Terms](#)

and

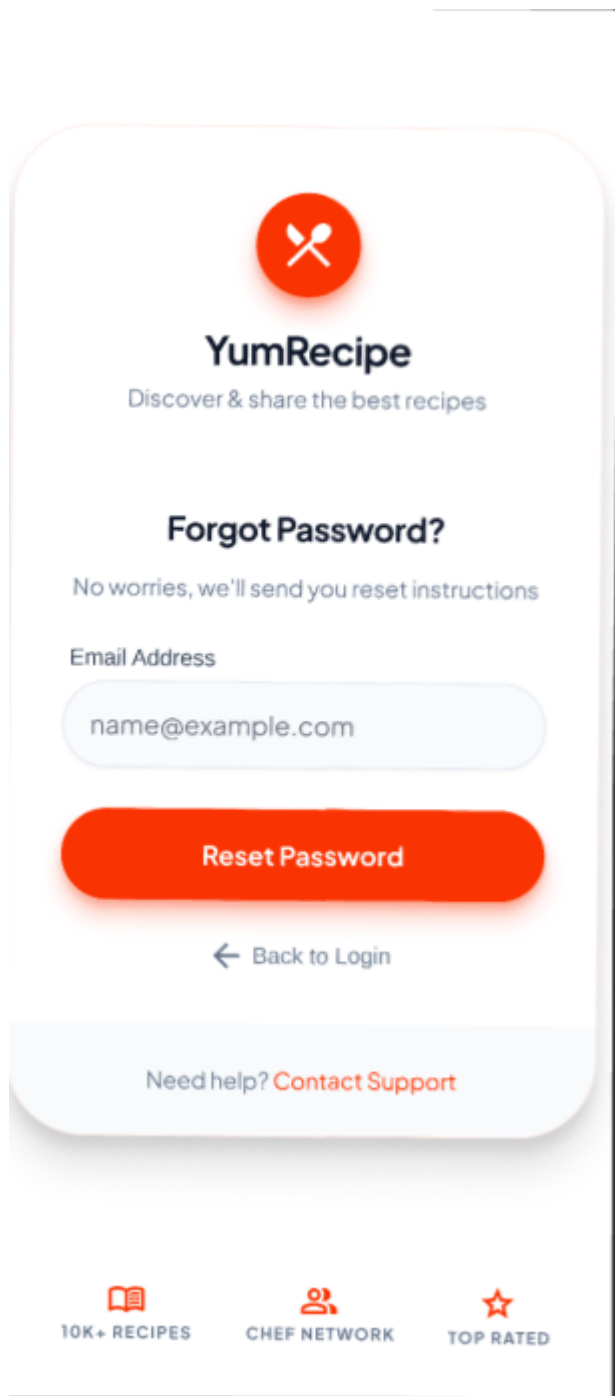
[Privacy Policy](#)

 10K+ RECIPES

 CHEF NETWORK

 TOP RATED

- Forgot password screen design:



- API implementation for user registration and login:

POST

/api/auth/register

Parameters

No parameters

Request body required

application/json

```
{
  "email": "lamhuukhanh2005@gmail.com",
  "password": "123456",
  "displayName": "Lâm Khánh"
}
```

Execute

Clear

Responses

Curl

curl -X 'POST' \
'http://localhost:8080/api/auth/register' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
 "email": "lamhuukhanh2005@gmail.com",
 "password": "123456",
 "displayName": "Lâm Khánh"
}'

Request URL

http://localhost:8080/api/auth/register

Server response

Code

Details

200

Response body

Đăng ký thành công!

Download

Response headers

connection: keep-alive
content-length: 24
content-type: text/plain; charset=UTF-8
date: Sat, 14 Mar 2026 11:33:21 GMT
keep-alive: timeout=60

Responses

Code

Description

Links

Auto-Commit ON - connected										
	id	email	password_hash	display_name	avatar_url	bio	is_active	is_banned	role	create
1	1	steph@hcmus.edu.vn	hashed_123	Stephrss05			true	false	ADMIN	Mar 14, 2026, 3
2	2	linh@example.com	hashed_abc	Linh Tran			true	false	USER	Mar 14, 2026, 3
3	3	an@example.com	hashed_def	An Nguyen			true	false	USER	Mar 14, 2026, 3
4	4	minh@example.com	hashed_ghi	Minh Le			true	false	USER	Mar 14, 2026, 3
5	5	hoa@example.com	hashed_jkl	Hoa Pham			true	false	USER	Mar 14, 2026, 3
6	6	khanh@example.com	hashed_mno	Khanh Vo			true	false	USER	Mar 14, 2026, 3
7	7	lamhuukhanh69@gmail.com	\$2a\$10\$DFEPs2gG0JGjzFpJkPQMweRCcKntHwosoDR/BwCWzuzInnHRIPAC	LK			true	false	USER	Mar 14, 2026, 3
8	8	lamhuukhanh2005@gmail.com	\$2a\$10\$1thFd48mVyRiqlL71vCgeeCS/YAzT/1Ysls6YWNdN6DseC5m4IK	Lâm Khánh			true	false	USER	Mar 14, 2026, 6

POST /api/auth/login

Parameters

No parameters

Request body required

application/json

```
{  "email": "lamhuukhanh2005@gmail.com",  "password": "123456"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "lamhuukhanh2005@gmail.com",
    "password": "123456"
  }'
```

Request URL

http://localhost:8080/api/auth/login

Server response

Code

Details

200

Response body

```
{  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJsYm1odXVraGuaDIwMDVAZ21hakMuY291IiwiaWF0IjoxNzgzNDg4MDU1LjCjIeHA10jE3NzMNzQ8NTV9.rtZjIZKCMaBfa-YpPK7a3MaNTjZ08GMZA3TLr26FqFI",  "email": "lamhuukhanh2005@gmail.com",  "displayName": "Lâm Khánh",  "avatarUrl": null,  "role": "USER"}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Sat, 14 Mar 2026 11:34:15 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
```

- API implementation for forgot password, email confirmation and password reset:

POST

/api/auth/forgot-password

Parameters

No parameters

Request body required

application/json

```
{
  "email": "lamhuukhanh2005@gmail.com"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/auth/forgot-password' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "lamhuukhanh2005@gmail.com"
  }'
```

Request URL

http://localhost:8080/api/auth/forgot-password

Server response

Code	Details
200	Response body Mã khôi phục đã được gửi đến email của bạn. Download Response headers connection: keep-alive content-length: 62 content-type: text/plain; charset=UTF-8 date: Sat, 14 Mar 2026 11:34:43 GMT keep-alive: timeout=60

Khôi phục mật khẩu - YumRecipe

Hộp thư đến x



daokinghacker05@gmail.com

đến tôi ▾

Mã khôi phục mật khẩu của bạn là: 261668

Mã này sẽ hết hạn sau 15 phút. Vui lòng không chia sẻ mã này cho bất kỳ ai.

POST

/api/auth/reset-password

Parameters

No parameters

Request body required

application/json

```
{  "email": "lamhuukhanh2005@gmail.com",  "resetToken": "261668",  "newPassword": "lamkhanhdeptrai"}}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://localhost:8080/api/auth/reset-password' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "email": "lamhuukhanh2005@gmail.com",  "resetToken": "261668",  "newPassword": "lamkhanhdeptrai"}'
```

Request URL

http://localhost:8080/api/auth/reset-password

Server response

Code	Details
200	Response body Mật khẩu đã được đặt lại thành công. Download Response headers connection: keep-alive content-length: 53 content-type: text/plain; charset=UTF-8 date: Sat, 14 Mar 2026 11:35:44 GMT

- Unit tests for authentication controllers and services:

server [test]: 10 total, 10 passed			1.48 s
			Collapse Expand
ServerApplicationTests			500 ms
contextLoads			passed 500 ms
AuthControllerTest			654 ms
testLogin_Success			passed 604 ms
testResetPassword_Success			passed 25 ms
testRegister_EmailExists			passed 9 ms
testRegister_Success			passed 8 ms
testForgotPassword_Success			passed 8 ms
JwtServiceTest			321 ms
testIsTokenValid			passed 311 ms
testExtractUsername			passed 4 ms
testGenerateToken			passed 4 ms
testIsTokenValid_IncorrectUser			passed 2 ms

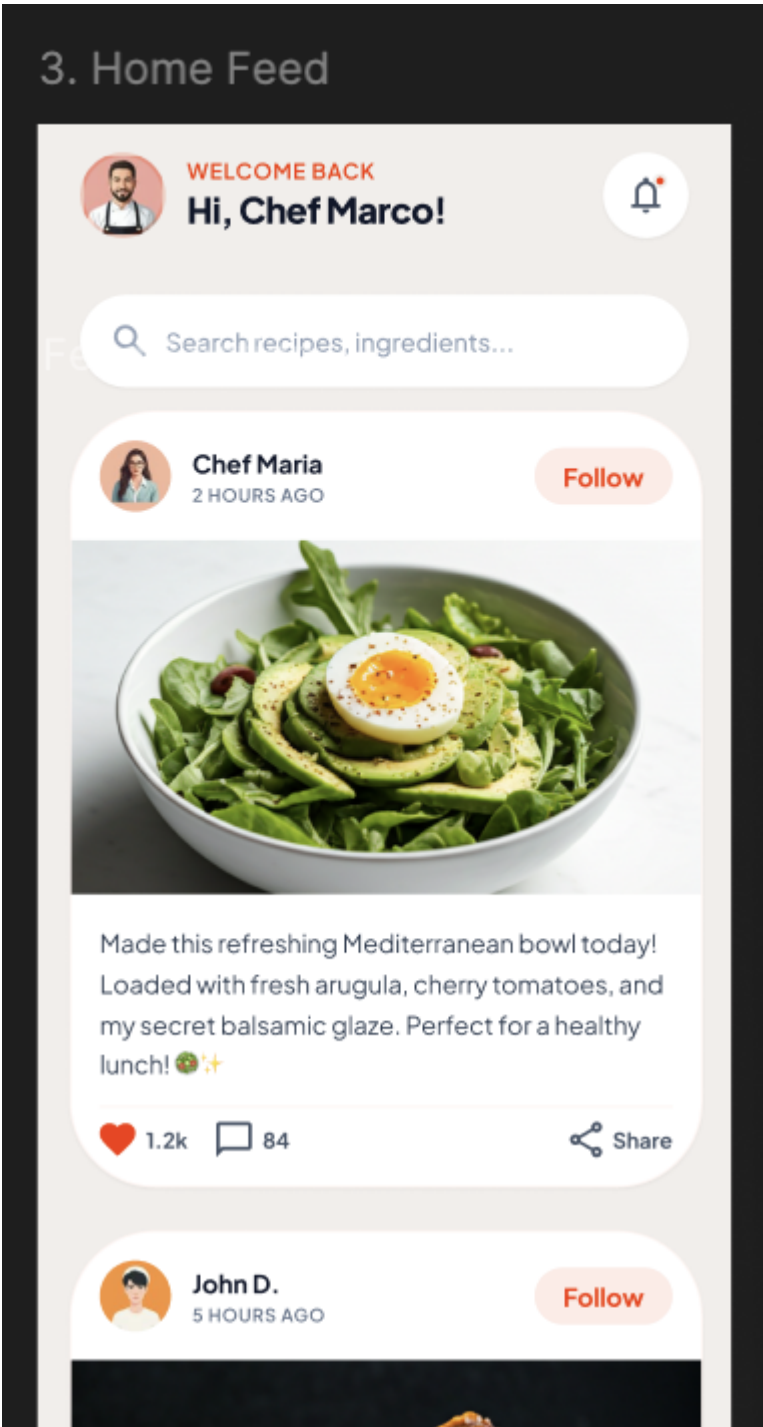
- Commit history for the implemented features:

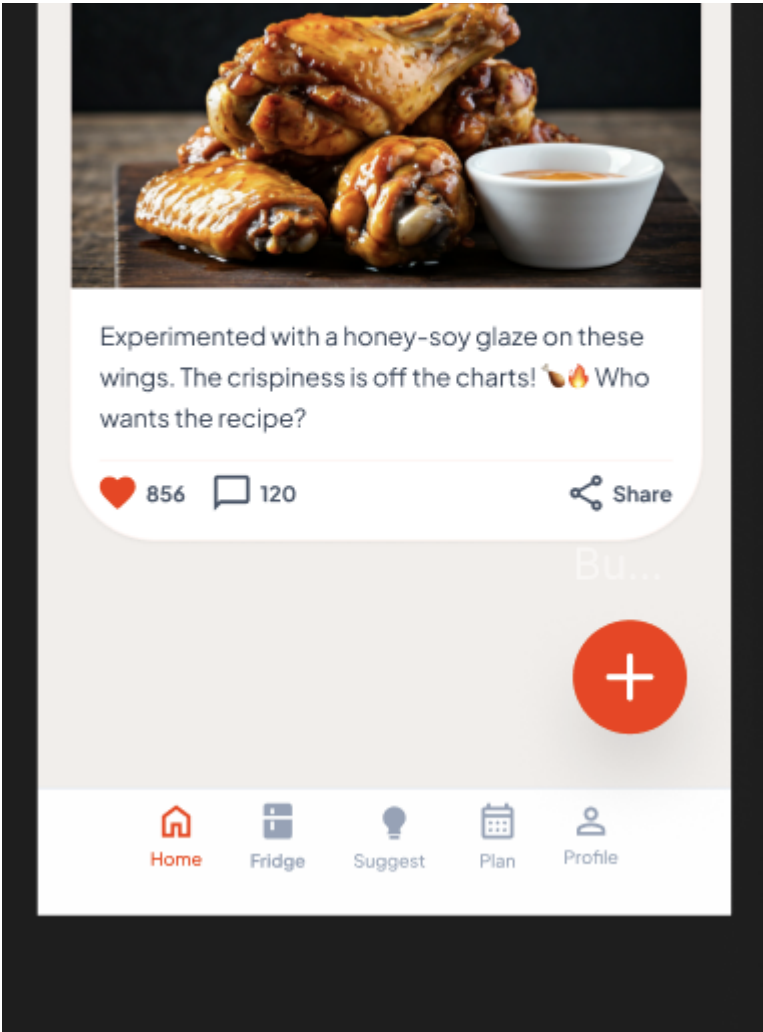
feat(android): implement authentication UI with login, register, forgot/reset password flows	0c31301		
test(server): add auth and JWT tests, fix timezone config for test environment	548ed15		
feat(server): implement authentication API with JWT and security config	493cd19		
chore: update .gitignore	4f219c5		

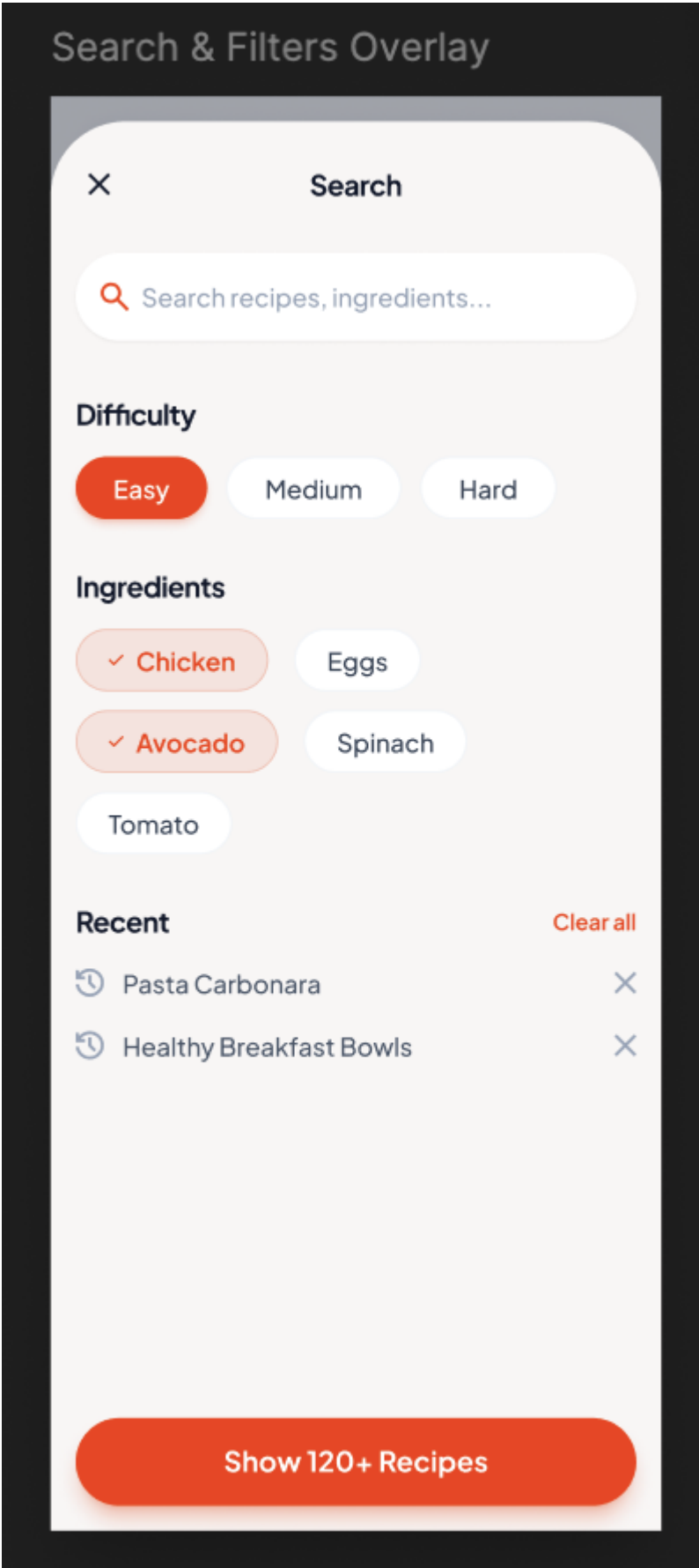
23127326 - Lê Mai Hoài Bảo

- Design the screen for the home feed, search, and search results.
- Implement the API to retrieve the list of recipes for the home feed and search results.
- Implement the API to retrieve the details of a specific recipe.

► Evidence







← Search Results



Vegetarian ▾

A close-up photograph of a white plate filled with spaghetti. The spaghetti is topped with a thick, creamy mushroom sauce. A single fresh basil leaf is placed on top of the sauce. The plate is set against a dark background.



★ 4.8

⌚ 20 min



★ 4.5

🕒 15 min



- API implementation for retrieving the list of recipes for the home feed and search results:

GET /api/recipes Search and filter recipes

Filter published recipes by title (partial), ingredient name(s), and/or difficulty. Supports pagination.

Parameters

Try it out

Name	Description
title	Partial recipe title (case-insensitive)
string (query)	title
ingredient	Ingredient keyword(s). Can be repeated: ?ingredient=garlic&ingredient=onion
array<string> (query)	
difficulty	Difficulty level: EASY, MEDIUM or HARD
string (query)	Available values : EASY, MEDIUM, HARD
	--
page	Zero-based page index (default: 0)
integer(\$int32) (query)	Default value : 0
	0
size	Number of results per page (default: 10)
integer(\$int32) (query)	Default value : 10
	10

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{  "success": true,  "message": "string",  "data": {    "content": [      {        "id": 9807199254748991,        "authorName": "string",        "title": "string",        "description": "string",        "difficulty": "EASY",        "status": "DRAFT",        "coverImageUrl": "string",        "createdAt": "2026-03-14T03:57:12.175Z",        "updatedAt": "2026-03-14T03:57:12.175Z"      }    ],    "pageNumber": 1073741824,    "pageSize": 1073741824,    "totalElements": 9807199254748991,    "totalPages": 1073741824,    "last": true  }  }
```

- API implementation for retrieving the details of a specific recipe:

GET /api/recipes/{id} Get recipe detail

Returns the full recipe including the list of ingredients and step-by-step cooking instructions.

Parameters

Name	Description
id * required integer(\$int64) (path)	Recipe ID id

Try it out

Responses

Code	Description	Links
200	OK	No links

Media type

*/

Controls Accept header.

Example Value | Schema

```
{
  "success": true,
  "message": "string",
  "data": {
    "id": 9007199254740991,
    "authorName": "string",
    "title": "string",
    "description": "string",
    "difficulty": "EASY",
    "status": "DRAFT",
    "coverImageUrl": "string",
    "ingredients": [
      {
        "name": "string",
        "quantity": 0.,
        "unit": "GRAM"
      }
    ],
    "steps": [
      "string"
    ],
    "createdAt": "2026-03-14T03:57:12.177Z",
    "updatedAt": "2026-03-14T03:57:12.177Z"
  }
}
```

- Commit history for the implemented features:

```
commit a7caddf590483f00340eea44636634d14d1eb1ea (HEAD -> main, origin/main, origin/HEAD)
Merge: e6dde4d 7d99786
Author: LE MAI HOAI BAO <152853155+HB4305@users.noreply.github.com>
Date: Sat Mar 14 11:01:08 2026 +0700

    Merge pull request #14 from lad94220/feature/search-filter-recipe

    feat: api-for-search-filter-recpie

commit 7d997860ad2665c0a48c9fd25874f9d9907cbf44 (origin/feature/search-filter-recipe, feature/search-filter-recipe)
Author: LE MAI HOAI BAO <152853155+HB4305@users.noreply.github.com>
Date: Thu Mar 12 20:13:06 2026 +0700

    feat: api for search/filter recpie
```

23127357 - Lê Anh Duy

- Create Figma design prototype using Stich and ChatGPT.
- Implementing fridge manager group feature (show, create).
- Implementing Gemini API config and normalize the response from it.
- Implementing the suggest strategy (request -> lookup current ingredients in the fridge -> lookup 20 match recipe -> ask Gemini for the best result -> normalize the response -> response the suggestion for user).

► Evidence

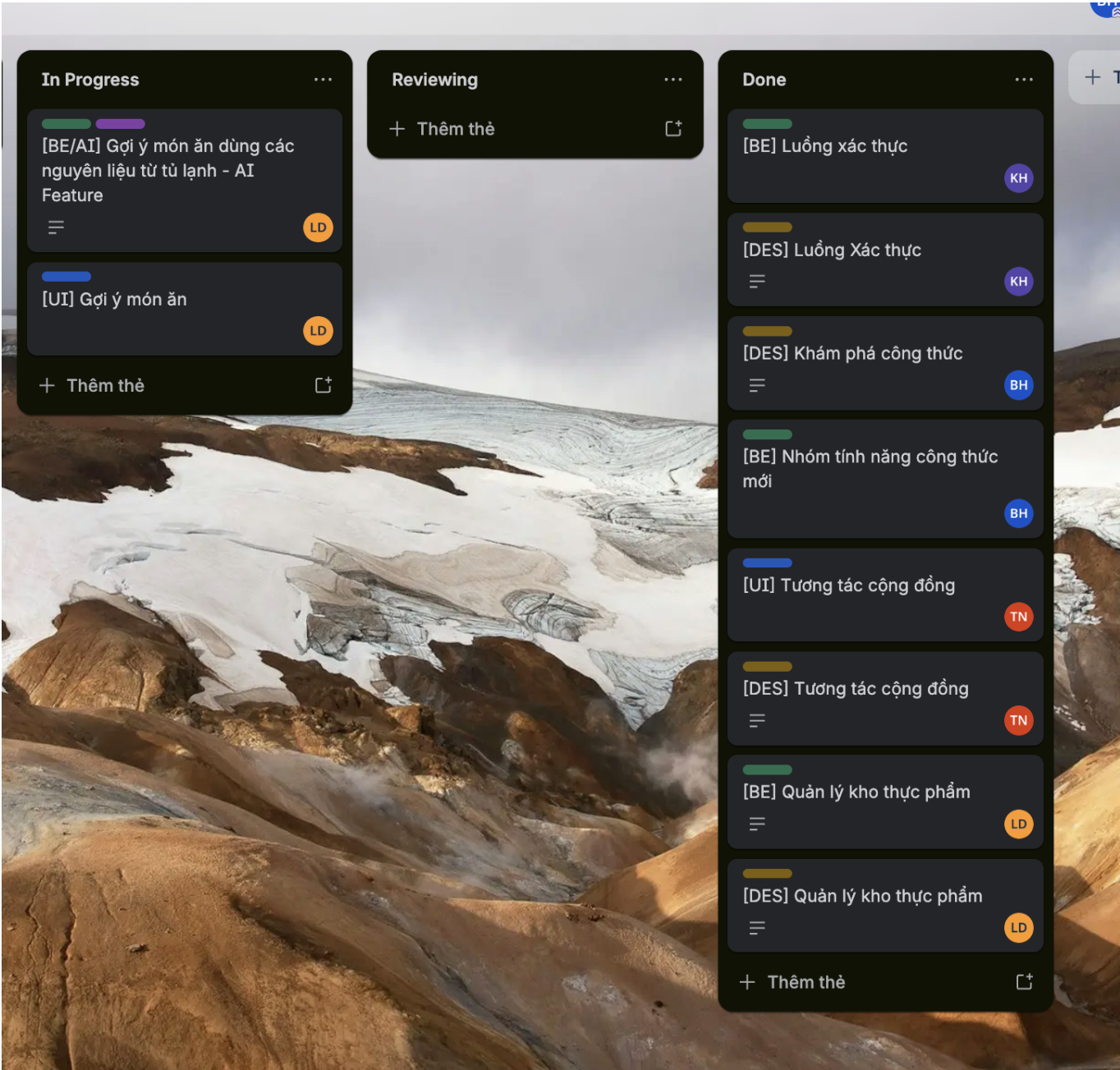
add navigation runtime package lad94220 committed 4 days ago	edfafa2	<>
add navigation runtime package version lad94220 committed 4 days ago	df526f3	<>
add fridge view to navigation bar lad94220 committed 4 days ago	2846e55	<>
add main fridge manager fragment lad94220 committed 4 days ago	679e780	<>
add fridge item ui component lad94220 committed 4 days ago	7c5d11c	<>
add fridge category ui component lad94220 committed 4 days ago	5c4f701	<>
add fridge fragment ui lad94220 committed 4 days ago	928031a	<>
add fridge model lad94220 committed 4 days ago	64581d3	<>
add fridge data binding for recyclerview lad94220 committed 4 days ago	321e3a3	<>
add runtime dependencies lad94220 committed 4 days ago	b0a7e5d	<>
fitting the navigation bar to the screen lad94220 committed 4 days ago	1a6aaaf	<>
add gemini api key properties lad94220 committed 4 days ago	bb14252	<>
add helper class lad94220 committed 4 days ago	6f0cb2e	<>
add recipe suggest res and req DTO lad94220 committed 4 days ago	73cf8ff	<>
add gemini api key properties lad94220 committed 4 days ago	4439896	<>
add helper class lad94220 committed 4 days ago	54a79a0	<>

add helper class lad94220 committed 4 days ago	54a79a0	<>
add recipe suggest res and req DTO lad94220 committed 4 days ago	5af4ff3	<>
Commits on Mar 12, 2026		
change the ui lad94220 committed 2 days ago	af83f57	<>
Merge branch 'main' of https://github.com/lad94220/cooking-recipe int... lad94220 committed 2 days ago	d04f8e8	<>
add api base url lad94220 committed 2 days ago	efb922a	<>
add retrofit setup lad94220 committed 2 days ago	547a3c4	<>
Merge branch 'feat/fridge-manager-ui' into feat/Al-suggestion lad94220 committed 2 days ago	043cd43	<>
Merge branch 'feat/Al-suggest-ui' into feat/Al-suggestion lad94220 committed 2 days ago	d746252	<>
delete unused title lad94220 committed 2 days ago	6b600fb	<>
Commits on Mar 14, 2026		
add fridge manager api endpoint lad94220 committed 39 minutes ago	339b4a8	<>
add fridge manager ui lad94220 committed 38 minutes ago	d64c653	<>
add adapter for fridge item list lad94220 committed 38 minutes ago	dab28ae	<>
add bottom sheet layout lad94220 committed 37 minutes ago	405a540	<>
config clear traffic lad94220 committed 37 minutes ago	7b774cb	<>

Commits on Mar 14, 2026		
add fridge manager api endpoint	lad94220 committed 39 minutes ago	339b4a8
add fridge manager ui	lad94220 committed 38 minutes ago	d64c653
add adapter for fridge item list	lad94220 committed 38 minutes ago	dab28ae
add bottom sheet layout	lad94220 committed 37 minutes ago	405a540
config clear traffic	lad94220 committed 37 minutes ago	7b774cb
add utils helper	lad94220 committed 36 minutes ago	d78d241
add retrofit client	lad94220 committed 36 minutes ago	40d0af9
add addition fridge model	lad94220 committed 35 minutes ago	fa9fc60
add fridge view fragment	lad94220 committed 34 minutes ago	4d95275
add create new item request model	lad94220 committed 34 minutes ago	5b92d5f
add fridge view model	lad94220 committed 33 minutes ago	ad7658a
add fridge type model	lad94220 committed 33 minutes ago	3d4bc5f
add fridge item type and image url column	lad94220 committed 32 minutes ago	62a2317
Commits on Mar 14, 2026		

Commits on Mar 14, 2026		
add maven repo milestone for openai	lad94220 committed 28 minutes ago	1739910
add mapper convert from separate item into grouped item list	lad94220 committed 26 minutes ago	d4b393a
add fridge item repository layer	lad94220 committed 26 minutes ago	447aa77
add service interface	lad94220 committed 26 minutes ago	442dd4b
add column to new current schema	lad94220 committed 25 minutes ago	19ae863
add configuration	lad94220 committed 25 minutes ago	373e82d
add fridge request dto	lad94220 committed 24 minutes ago	496e26d
add fridge response	lad94220 committed 24 minutes ago	985c2ba
add fridge manager api endpoint controller	lad94220 committed 24 minutes ago	42ff79d
add fridge manager service implementation	lad94220 committed 23 minutes ago	13f93dc

Trello Board for Task Management:



AI Usage Declaration

23127205 - Lâm Hữu Khánh

1. General Information

- **Tool name, version, and platform:** Gemini 3 Pro gemini.google.com.
- **Access time:** 17:00 March 14, 2026.

2. Usage Details

- **Prompts used:**
 - "Implement Spring Boot APIs for User Registration and Secure Login using JWT (JSON Web Tokens)."
 - "Create an Email Service to handle Forgot Password and Password Reset requests."
- **Purpose of use:** To generate boilerplate code for security configurations

3. Content Declaration

- **Which content was generated by AI:**
 - Base structure for `AuthController.java`, `AuthService.java`, and JWT security configuration.
 - Core logic for the Email Service and password reset token handling.
- **Which content was done independently and how I edited and validated it:**
 - **Integration:** Integrated the JWT logic with the custom `UserDetails` and Database schema.
 - **Security Logic:** Independently verified and strengthened the security measures, such as password encryption and token expiration.
 - **Technical Verification:** Successfully compiled the project and tested the end-to-end authentication flow on the Android client.

4. Evidence

- **Prompts:**

Implement Spring Boot APIs for User Registration and Secure Login using JWT (JSON Web Tokens)

To implement a secure authentication system in Spring Boot using JWT, we need to configure **Spring Security**, handle user persistence, and create a utility class for token generation and validation.

Below is the structured implementation for your **User Registration** and **Secure Login** flow.

1. Security Dependencies

Add these to your `pom.xml` to include Spring Security and the JJWT library for handling tokens.

XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

2. JWT Utility Class (`JwtTokenProvider`)

This class handles the creation, parsing, and validation of the JWT.

Java

```
@Component
public class JwtTokenProvider {

    private final String JWT_SECRET = "23127205_Your_Super_Secret_Key_For_Signing";
    private final long JWT_EXPIRATION = 86400000L; // 24 hours

    public String generateToken(Authentication authentication) {
        String username = authentication.getName();
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + JWT_EXPIRATION);

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(expiryDate)
            .signWith(Keys.hmacShaKeyFor(JWT_SECRET.getBytes()), SignatureAlgorithm.HMACSHA256)
            .compact();
    }

    public String getUsernameFromJWT(String token) {
        Claims claims = Jwts.parserBuilder()
            .setSigningKey(Keys.hmacShaKeyFor(JWT_SECRET.getBytes()))
            .build()
            .parseClaimsJws(token)
            .getBody();
        return claims.getSubject();
    }
}
```

3. Authentication Controller

This is the entry point for your API. It handles the user's request to register or log in.

Java

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private JwtTokenProvider tokenProvider;

    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@RequestBody SignUpRequest signUpRequest) {
        if(userRepository.existsByUsername(signUpRequest.getUsername())) {
            return new ResponseEntity<>("Username is already taken!", HttpStatus.BAD_REQUEST);
        }

        User user = new User();
        user.setUsername(signUpRequest.getUsername());
        // Crucial: Always encode passwords before saving
        user.setPassword(passwordEncoder.encode(signUpRequest.getPassword()));

        userRepository.save(user);
    }
}
```



```

        userRepository.save(user);
        return ResponseEntity.ok("User registered successfully");
    }

    @PostMapping("/login")
    public ResponseEntity<?> authenticateUser(@RequestBody LoginRequest loginReq) {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                loginRequest.getUsername(),
                loginRequest.getPassword()
            )
        );

        SecurityContextHolder.getContext().setAuthentication(authentication);
        String jwt = tokenProvider.generateToken(authentication);

        return ResponseEntity.ok(new JwtAuthenticationResponse(jwt));
    }
}

```

4. Security Configuration (`SecurityConfig`)

This configures how Spring Security protects your routes.

Java



```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
    }
}

```

Core Security Principles Applied:

- **Password Encoding:** Uses `BCryptPasswordEncoder` to ensure passwords aren't stored in plain text.
- **Statelessness:** No sessions are stored on the server; the JWT contains all necessary info.
- **Token Validation:** Every protected request must pass through a filter (omitted here for brevity but required in production) to verify the `Authorization: Bearer <token>` header.

Would you like me to generate the `JwtAuthenticationFilter` code to complete the request-response verification loop?

Create an Email Service to handle Forgot Password and Password Reset requests.

To build a robust "Forgot Password" flow, we need a service that generates a secure, time-limited token, stores it in the database, and dispatches an email to the user.

Below is the implementation using **Spring Boot Starter Mail** and a repository-based token management system.

1. Password Reset Token Entity

First, we need a way to store the token and link it to a user.

Java

```
@Entity
public class PasswordResetToken {
    private static final int EXPIRATION = 60 * 24; // 24 hours

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String token;

    @OneToOne(targetEntity = User.class, fetch = FetchType.EAGER)
    @JoinColumn(nullable = false, name = "user_id")
    private User user;

    private Date expiryDate;

    public PasswordResetToken(String token, User user) {
        this.token = token;
        this.user = user;
        this.expiryDate = calculateExpiryDate(EXPIRATION);
    }

    private Date calculateExpiryDate(int expiryTimeInMinutes) {
        Calendar cal = Calendar.getInstance();
        cal.setTime(new Timestamp(cal.getTime().getTime()));
        cal.add(Calendar.MINUTE, expiryTimeInMinutes);
        return new Date(cal.getTime().getTime());
    }

    // Getters and Setters...
}
```

This service handles the actual sending of the email via SMTP.

Java

```
@Service
public class EmailService {
```

```

@Autowired
private JavaMailSender mailSender;

public void sendResetTokenEmail(String contextPath, String token, User user) {
    String url = contextPath + "/api/auth/changePassword?token=" + token;
    String message = "To reset your password, click the link below:\n" + url;

    SimpleMailMessage email = new SimpleMailMessage();
    email.setSubject("Reset Password - Project 23127205");
    email.setText(message);
    email.setTo(user.getEmail());
    email.setFrom("support@yourapp.com");

    mailSender.send(email);
}
}

```

3. Password Reset Logic in AuthService

The service layer coordinates finding the user, generating a unique `UUID` token, and triggering the email.

```

Java
@Service
public class AuthService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordResetTokenRepository tokenRepository;

    @Autowired
    private EmailService emailService;

    public void createPasswordResetTokenForUser(String email, String contextPath) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UserNotFoundException("Email not found"));

        String token = UUID.randomUUID().toString();
        PasswordResetToken myToken = new PasswordResetToken(token, user);
        tokenRepository.save(myToken);

        emailService.sendResetTokenEmail(contextPath, token, user);
    }

    public boolean validatePasswordResetToken(String token) {
        PasswordResetToken passToken = tokenRepository.findByToken(token);
        return passToken != null && !passToken.getExpiryDate().before(new Date());
    }
}

```

4. API Endpoints

Finally, expose the functionality to the frontend.

Java

```
@RestController
@RequestMapping("/api/auth")
public class PasswordController {

    @Autowired
    private AuthService authService;

    @PostMapping("/forgotPassword")
    public ResponseEntity<?> resetPassword(HttpServletRequest request, @RequestParam
        String contextPath = request.getScheme() + "://" + request.getServerName() + request.getContextPath();
        authService.createPasswordResetTokenForUser(userEmail, contextPath);
        return ResponseEntity.ok("Reset link sent to your email.");
    }

    @PostMapping("/savePassword")
    public ResponseEntity<?> savePassword(@RequestBody PasswordDto passwordDto) {
        if (!authService.validatePasswordResetToken(passwordDto.getToken())) {
            return ResponseEntity.badRequest().body("Invalid or Expired Token");
        }

        authService.updateUserPassword(passwordDto.getToken(), passwordDto.getNewPassword());
        return ResponseEntity.ok("Password updated successfully.");
    }
}
```

Security Checkpoint:

- **Token Uniqueness:** Using `UUID.randomUUID()` ensures high entropy.
- **Expiration:** The `expiryDate` check prevents old links from being exploited.
- **One-time Use:** In a production environment, you should delete the token from the database immediately after a successful password change.

Would you like me to write the Unit Tests for this Email Service using Mockito next?

23127122 - Nguyễn Duy Thịnh

1. General Information

- **Tool name, version, and platform:** Stitch (Gemini 3.1 Pro Preview), Anthropic Claude Sonet 4.6.
- **Access time:** From 08/03/2026 to 12/03/2026.

2. Usage Details

- **Prompts used:**
- Using Stitch to generate the UI design for the post view and sharing view.



Thinh Nguyen

Can you elaborate based on the community feed, like when I click on a post, show a screen with post and comments (inspired by X app), also when click share show a view where there's a few option (export to pdf, create canva image) and the center big is the QR code

- Using Claude to brainstorm how to implement the feed feature group (the context of the project is already provided).

following mvvm pattern, for feed view, can you suggest file structure and what each does in that structure?

Files



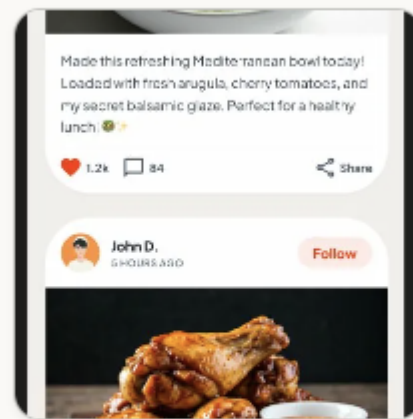
1% of project capacity used

Đề tài project

34 lines

TEXT

- Using Claude to create the XML for the feed view.



can you create the xml for this feed?

- **Purpose of use:**
- Using AI to generate UI design for the post view and sharing view as a baseline for UX.
- Using AI to easily understand how one would implement using MVVM pattern.
- Using AI to generate UI design and XML layout for the feed feature group efficiently.

3. Content Declaration

- **Which content was generated by AI:**
 - Mock data for the home feed screen.
 - XML layout for the home feed screen and postcard (`fragment_feed.xml`, `item_feed_post.xml`)

23127326 - Lê Mai Hoài Bảo

1. General Information

- **Tool name, version, and platform:** Gemini 3.1 Pro (Antigravity model), Google, gemini.google.com.
- **Access time:** 13:18 on March 14, 2026[cite: 82].

2. Usage Details

- **Prompts used:** "Tìm kiếm và lọc công thức (theo tên, nguyên liệu, độ khó). Xem danh sách nguyên liệu và quy trình chế biến. Hãy viết API cho mấy cái trên."
- **Purpose of use:** To design the implementation plan and generate the class structures (DTOs, Repository, Specification, Service, and Controller) for the Recipe module in the "Yum Recipe" project.

3. Content Declaration

- **Which content was generated by AI:**

- The architectural structure for `RecipeDTO.java`, `RecipeDetailDTO.java`, `RecipeRepository.java`, `RecipeSpecification.java`, `RecipeService.java`, and `RecipeController.java`.
- Logic for dynamic filtering using Spring Data JPA Specifications and PostgreSQL JSONB substring matching.
- Sample API request/response shapes and curl commands.
- **Which content was done independently and how I edited and validated it:**
 - **Integration:** Incorporated the AI-generated logic into the existing project framework to match `ApiResponse<T>` and `PageResponse<T>` classes.
 - **Unit Testing:** Independently developed and executed Unit Tests using JUnit/Mockito to verify business logic and exception handling.
 - **Validation (Integration Testing):** Used **Swagger UI** (`/swagger-ui.html`) to perform manual integration testing with a real database to confirm filter accuracy.
 - **Technical Verification:** Executed `./gradlew compileJava` to ensure code compatibility and syntax correctness.

4. Evidence

- **Prompts:**

```

Tìm kiếm và lọc công thức (theo tên, nguyên liệu, độ khó)
Xem danh sách nguyên liệu và quy trình chế biến

Hãy viết API cho mấy cái trên

> Thought for 2s

Analyzed server
Analyzed server/src/main/java/com/yum_recipe/server

Planning Recipe Search & Filter API
Fully explored project structure: 📁 Recipe entity with title, difficulty (enum), status, ingredients (JSONB), steps (JSONB). Existing patterns: 📁 ApiResponse, 📁 PageResponse, 📁 GlobalExceptionHandler. The recipe package is completely empty.

Files Edited
📄 Implementation Plan

Progress Updates
Expand all <
1 Exploring project structure and existing entities
2 Reading remaining relevant files and existing test structure
3 Checking for existing tests and build configuration, then writing implementation plan
4 Writing implementation plan artifact

Tôi đã khám phá codebase và lập kế hoạch triển khai. Vui lòng xem và xác nhận plan trước khi tôi bắt đầu code.
Tóm tắt những gì sẽ được tạo ra (tất cả trong package recipe):

• RecipeDTO — DTO nhẹ cho danh sách kết quả tìm kiếm
• RecipeDetailDTO — DTO đầy đủ gồm ingredients và steps
• RecipeRepository — Spring Data JPA + JpaSpecificationExecutor
• RecipeSpecification — Dynamic filter bằng JPA Specifications (title, difficulty, ingredient)
• RecipeService — Business logic
• RecipeController — 2 endpoints:

  • GET /api/recipes?title=...&ingredient=...&difficulty=... → Tìm kiếm & lọc (có phân trang)
  • GET /api/recipes/{id} → Xem nguyên liệu và các bước chế biến

```

Bạn có muốn tôi tiếp tục không?

Implementation Plan

Open

Plan to implement Recipe Search & Filter API (by title, ingredient, difficulty) and Recipe Detail API (ingredients + steps). Creates 5 new files in the recipe package: RecipeDTO, RecipeDetailDTO, RecipeRepository, RecipeSpecification, RecipeService, RecipeController. Follows existing project patterns (ApiResponse, PageResponse, GlobalExceptionHandler). Includes build verification and manual testing steps via Swagger UI.

Copy

Proceeded with Implementation Plan

Auto-proceeded by agent.

Always Proceeds

Thought for 10s

Implementing Recipe APIs

All 6 files created and verified via code review. `compileJava` via Gradle failed due to Java 21 toolchain not detected locally — this is a local JDK config issue, not a code error. All files are correctly structured.

Files Edited

Walkthrough RecipeController.java RecipeService.java RecipeSpecification.java RecipeRepository.java RecipeDetailDTO.java +1 more

Progress Updates

Expand all

1 Creating all files in the recipe package

2 Creating RecipeRepository, RecipeSpecification, RecipeService, RecipeController

3 Running Gradle compilation to verify all files compile correctly

4 Writing walkthrough and notifying user of completion

Đã tạo xong toàn bộ API! 6 file mới trong package `recipe` :

2 Endpoints:

GET /api/recipes — Tìm kiếm & lọc (theo tên, nguyên liệu, độ khó), có phân trang

GET /api/recipes/{id} — Xem chi tiết: danh sách nguyên liệu + các bước chế biến

Lưu ý nhỏ: Khi chạy `./gradlew compileJava` bị lỗi `"Cannot find Java 21 toolchain"` — đây là vấn đề cấu hình JDK trên máy, không liên quan đến code. Bạn chỉ cần đảm bảo `JAVA_HOME` trỏ đúng vào JDK 21 là chạy được bình thường.

Walkthrough

Open

Walkthrough of Recipe Search/Filter and Detail API implementation. Documents all 6 files created, the API endpoints, example requests, and a note about Gradle compile verification.

Copy

- **Testing Evidence:**
 - Unit tests for the Recipe module:

All Results

Gradle Test Run :test

Gradle Test Run :test

summary

5 tests

0 failures

0 skipped

1.599s duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
RecipeControllerTest	com.yum_recipe.server.recipe.RecipeControllerTest	2	0	0	1.031s	100%
RecipeServiceTest	com.yum_recipe.server.recipe.RecipeServiceTest	3	0	0	0.097s	100%

31 / 36

- Integration testing using Swagger UI:

Curl

curl -X 'GET' \
'http://localhost:8080/api/recipes?title=pasta&page=0&size=10' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/recipes?title=pasta&page=0&size=10

Server response

Code

Details

200

Response body

```
{  
  "success": true,  
  "message": "Success",  
  "data": {  
    "content": [  
      {  
        "authorName": "Linh Tran",  
        "coverImageUrl": null,  
        "createdAt": "2026-03-12T13:00:11.616427Z",  
        "description": "Simple weeknight pasta with garlic and butter.",  
        "difficulty": "EASY",  
        "id": 1,  
        "status": "PUBLISHED",  
        "title": "Garlic Butter Pasta",  
        "updatedAt": "2026-03-12T13:00:11.616427Z"  
      }  
    ],  
    "pageNumber": 0,  
    "pageSize": 10,  
    "totalElements": 1,  
    "totalPages": 1,  
    "last": true  
  }  
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate  
connection: keep-alive  
content-type: application/json  
date: Sat,14 Mar 2026 05:58:11 GMT  
expires: 0  
keep-alive: timeout=60  
pragma: no-cache  
transfer-encoding: chunked  
x-content-type-options: nosniff  
x-frame-options: DENY  
x-xss-protection: 0
```

Curl

curl -X 'GET' \
'http://localhost:8080/api/recipes?difficulty=EASY&page=0&size=10' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/recipes?difficulty=EASY&page=0&size=10

Server response

Code

Details

200

Response body

```
{  
  "success": true,  
  "message": "Success",  
  "data": {  
    "content": [  
      {  
        "authorName": "Linh Tran",  
        "coverImageUrl": null,  
        "createdAt": "2026-03-12T13:00:11.616427Z",  
        "description": "Simple weeknight pasta with garlic and butter.",  
        "difficulty": "EASY",  
        "id": 1,  
        "status": "PUBLISHED",  
        "title": "Garlic Butter Pasta",  
        "updatedAt": "2026-03-12T13:00:11.616427Z"  
      },  
      {  
        "authorName": "Minh Le",  
        "coverImageUrl": null,  
        "createdAt": "2026-03-12T13:00:11.616427Z",  
        "description": "Quick breakfast omelette with fresh tomato.",  
        "difficulty": "EASY",  
        "id": 3,  
        "status": "PUBLISHED"  
      }  
    ]  
  }  
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate  
connection: keep-alive  
content-type: application/json  
date: Sat,14 Mar 2026 06:02:16 GMT  
expires: 0  
keep-alive: timeout=60  
pragma: no-cache  
transfer-encoding: chunked  
x-content-type-options: nosniff  
x-frame-options: DENY  
x-xss-protection: 0
```


Curl

curl -X 'GET' \
'http://localhost:8080/api/recipes?ingredient=garlic&ingredient=onion&page=0&size=10' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/recipes?ingredient=garlic&ingredient=onion&page=0&size=10

Server response

Code

Details

200

Response body

{
 "success": true,
 "message": "Success",
 "data": {
 "content": [],
 "pageNumber": 0,
 "pageSize": 10,
 "totalElements": 0,
 "totalPages": 0,
 "last": true
 }
}

Download

Response headers

cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Sat,14 Mar 2026 06:19:34 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

Curl

curl -X 'GET' \
'http://localhost:8080/api/recipes?title=soup&ingredient=tomato&difficulty=EASY&page=0&size=5' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/recipes?title=soup&ingredient=tomato&difficulty=EASY&page=0&size=5

Server response

Code

Details

200

Response body

{
 "success": true,
 "message": "Success",
 "data": {
 "content": [],
 "pageNumber": 0,
 "pageSize": 5,
 "totalElements": 0,
 "totalPages": 0,
 "last": true
 }
}

Download

Response headers

cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Sat,14 Mar 2026 06:28:40 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

Curl

curl -X 'GET' \
'http://localhost:8080/api/recipes/1' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/recipes/1

Server response

Code

Details

200

Response body

{
 "success": true,
 "message": "Success",
 "data": {
 "authorName": "Linh Tran",
 "coverImageUrl": null,
 "createdAt": "2026-03-12T13:00:11.616427Z",
 "description": "Simple weeknight pasta with garlic and butter.",
 "difficulty": "EASY",
 "id": 1,
 "ingredients": [
 {
 "name": "Spaghetti",
 "quantity": 200,
 "unit": "GRAM"
 },
 {
 "name": "Garlic",
 "quantity": 4,
 "unit": "PIECE"
 },
 {
 "name": "Butter",
 "quantity": 2,
 "unit": "TABLESPOON"
 }
]
 }
}

Download

Response headers

cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Sat,14 Mar 2026 06:29:05 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

Responses

- Compilation success:

```
(base) HB4305 ~/KhoChua/DH/Nam3/HK2/MOBILE/cooking-recipe/server % ./gradlew compileJava

BUILD SUCCESSFUL in 769ms
1 actionable task: 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.1/userguide/configuration_cache_enabling.html
(base) HB4305 ~/KhoChua/DH/Nam3/HK2/MOBILE/cooking-recipe/server %
```

23127357 - Lê Anh Duy

1. General Information

- **Tool name, version, and platform:** Gemini 3.1 Pro Preview, ChatGPT 5.1, Stich (Gemini 3.1 Pro Preview), Anthropic Claude Sonet 4.6.
- **Access time:** From 08/03/2026 to 13/03/2026.

2. Usage Details

- **Prompts used:**
- Using Chat GPT to generate the instruction for Stich

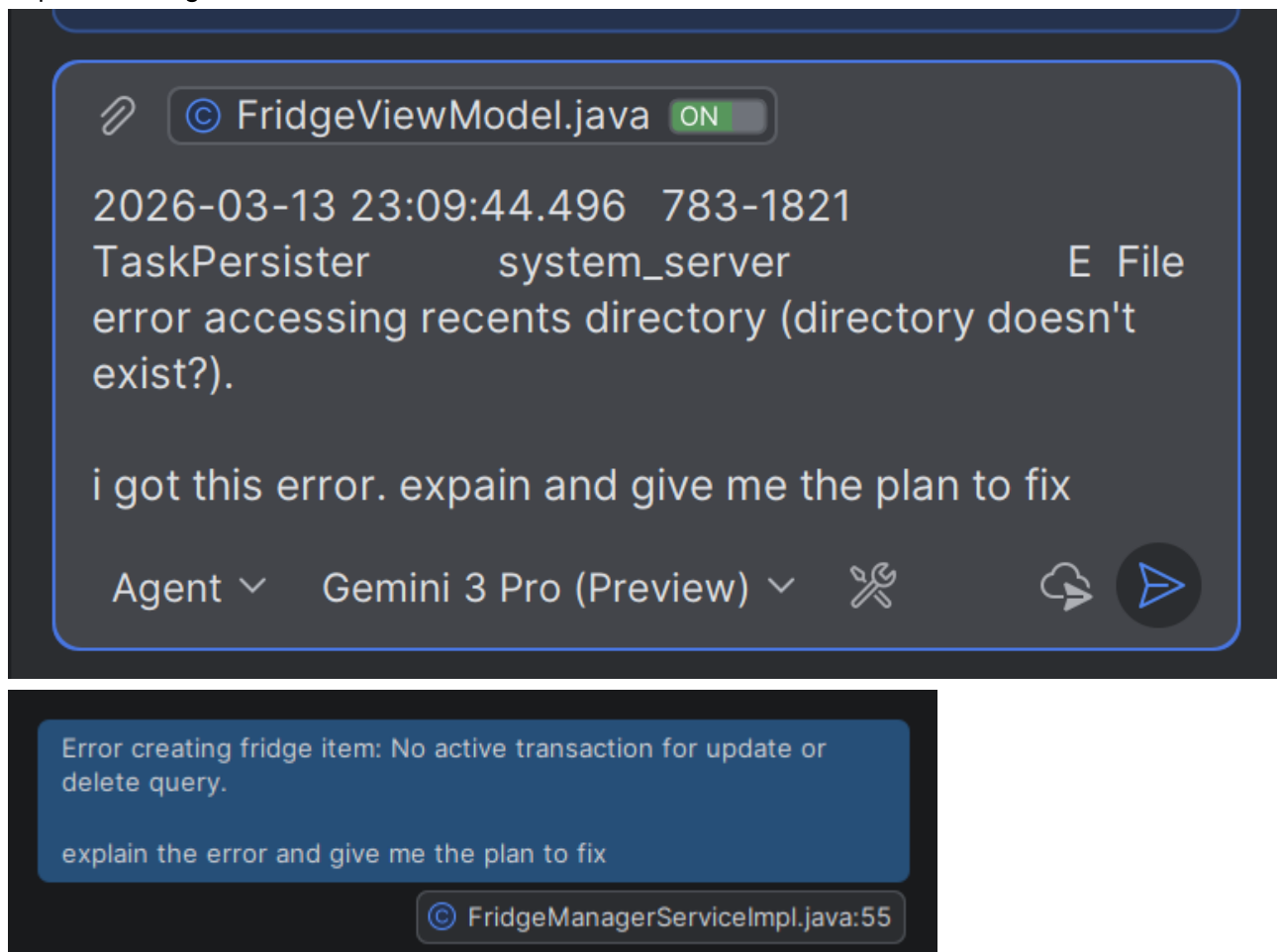
as a senior prompt engineer, please convert this into a prompt for stitch, which is an agent to generate ui, before generate please research about the palette from the similar application for me, i would like to have a aesthetic ui and friendly ux mobile application.

- Using Chat GPT to brainstorm and explore how can i implement the work flow

there is a task like this: android will pickup the image => save somewhere safe in phone storage => get the uuid => combine with other information in request => send to server

how can i implement the pickup, and save process

- Explain the bug



- **Purpose of use:**
- Using for generate UI design, brainstorm workflow, and support to code quicker.
- Using AI to generate UI layer efficiently and save the time.

3. Content Declaration

- **Which content was generated by AI:**
 - The UI layout: `component_item_fridge_category.xml`, `component_item_ingredient_card.xml`, `component_suggestion_list_item.xml`, `fragment_add_fridge_item.xml`, `fragment_fridge_manager.xml`, and `RecipeController.java`.

Tasks Planned for Next Week

23127122 - Nguyễn Duy Thịnh

- Implement the API for retrieving the post details, comments, and likes.
- Implement the API for creating a new post and sharing recipes.
- Implement the API for following feed.
- Binding data from the server in replacement of dummy data for the home feed screen.

23127205 - Lâm Hữu Khánh

- Complete UI for login, register, and forgot password.
- Design screen and API for profile (Account Management):
- API for Edit personal information.
- API for Account settings (Private/Public).

23127326 - Lê Mai Hoài Bảo

- Design the screen for the recipe details.
- Design the screen for creating and editing a recipe.
- Implement the API to create a new recipe.
- Implement the API to edit an existing recipe.
- Implement the API to delete a recipe.
- Implement the feature to view recipe details (ingredients, quantities, etc.).
- Develop hands-free cooking mode using SpeechRecognizer API.
- Add timer functionality for cooking steps.

23127357 - Lê Anh Duy

- Binding data from AI recipe suggest API to the frontend using view model.
- Enhance the UI of the AI recipe suggest view and add the delete fridge item popup.
- Finish the suggest recipe feature.